BriskMoE: Effective Speculative Decoding for Offloaded Mixture-of-Experts Inference on a Single GPU

Anonymous Author(s)

ABSTRACT

CPU offloading enables commodity-GPU serving of Mixture-of-Experts (MoE) models, but efficient decoding then critically depends on expert caching. Since speculative decoding (SD) delivers substantial speedups on dense models, it is a natural candidate for further accelerating cache-assisted offloaded MoE decoding. However, SD performs worse than expected: multi-token verification substantially expands the per-layer expert working set, yet even when the cache is large enough to hold SD's steady-state working set, existing AR-optimized cache designs fail to translate this into proportional speedup. We analyze the interaction between cache capacity and SD effectiveness, and derive a memory threshold M^* that marks a phase transition: below M^*, SD provides no benefit regardless of system optimizations; above M^*, two practical barriers still prevent SD from reaching its full potential, namely overhead on the cache-hit path and transient working-set overflow that triggers cascading evictions. To address these barriers, we present BriskMoE, a system with two complementary designs: 1) *Unblock* restores an HBM-speed cache-hit path by enabling direct execution on cache-resident experts, reusing stable expert-to-slot mappings on the common path, and overlapping residual misses with computation; 2) *Protect* pre-stages likely verification experts during draft generation, preventing transient working-set bursts from cascading into sustained cache misses. Extensive experiments on Qwen3-30B-A3B with EAGLE-3 show that BriskMoE achieves up to 4.80× speedup over the offload baseline, increasing decode throughput from 2.08 to 9.98 tok/s.

KEYWORDS

Mixture-of-Experts inference, speculative decoding, CPU offloading, GPU memory hierarchy, vLLM

1 INTRODUCTION

Latency-first, single-request serving is the dominant mode for interactive LLM applications such as chat assistants, code completion, and enterprise knowledge retrieval. Mixture-of-Experts (MoE) models are increasingly adopted in this setting [6, 11, 18], as they scale model capacity without proportionally increasing per-token compute; frontier open-weight models such as Qwen3, Mixtral, and DeepSeek-V3 are all MoE architectures. However, the total expert memory of these models exceeds the GPU budget, necessitating CPU offloading. Recent systems rely on GPU-resident expert caching to reduce PCIe traffic and accelerate autoregressive (AR) decoding [4, 8, 21]. Speculative decoding (SD) is the natural next candidate for further reducing decode latency, as it routinely delivers 2–3× speedup on dense models [1, 12, 14, 15].

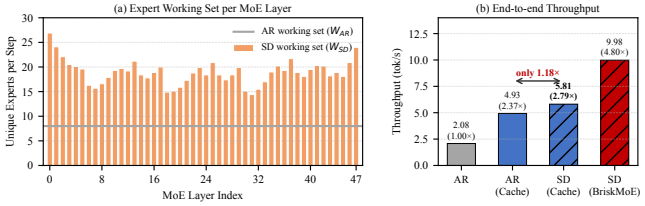

Figure 1: Motivating profiling on Qwen3-30B-A3B at 44 GiB. (a) SD expands the per-layer expert working set from $W_{\text{AR}} = 8$ to an average of $W_{\text{SD}} \approx 19.3$ (up to 26.9). (b) End-to-end throughput: adding SD on top of an MoE-Infinity expert cache yields only 1.18× speedup over cached AR (4.93→5.81 tok/s); BriskMoE reaches 9.98 tok/s (4.80× over the offload baseline).

A central question is whether SD remains equally effective in cached, offloaded MoE inference.

To investigate, consider a developer running a chat assistant powered by Qwen3-30B-A3B on a workstation equipped with a single 46 GB GPU. The model has 128 experts per layer, requiring about 57.6 GB in bf16, which exceeds the GPU budget, so expert weights are offloaded to host memory. With an MoE-Infinity-style expert cache [21], AR decoding already reaches 4.93 tok/s, a 2.37× speedup over uncached offloading. We then add EAGLE-3 speculative decoding at speculation depth K=3, which serves as the default draft setting throughout the paper. As Figure 1b shows, however, adding SD yields only 1.18× additional speedup (4.93→5.81 tok/s), far below the 2–3× seen on dense models. Figure 1a illustrates the underlying challenge: SD's multi-token verification expands the per-layer expert working set from $W_{\text{AR}} = 8$ under AR to an average of $W_{\text{SD}} \approx 19.3$, substantially increasing the number of expert accesses the cache must serve each step. Yet the bottleneck is not simply that the cache is too small: under the same cache configuration, BriskMoE reaches 9.98 tok/s (4.80×). Since both systems use identical expert cache capacity, the problem lies in *how* the existing cache design handles the expanded working set, not in the cache size itself. Section 3 profiles this phenomenon and identifies the specific cache-path inefficiencies responsible.

Existing works cannot effectively address this problem, primarily due to three gaps. First, batch-oriented analyses assume that each layer loads the full expert set, making SD unconditionally beneficial through improved arithmetic intensity [20]; this does not hold for a single request, where each layer loads only the top-k activated experts and SD's expanded working set manifests as additional PCIe traffic. Second, cache-based MoE systems such as MoE-Infinity and

pre-gated MoE design their cache policies exclusively for AR decoding and do not address the efficiency challenges that arise when SD's wider per-layer expert set stresses the cache access path [4, 8, 21]. Third, no prior work quantifies the conditions under which SD can benefit from expert caching, leaving the relationship between cache capacity, working-set size, and SD effectiveness uncharacterized. To understand when and why SD underperforms, and under what conditions it can succeed, Section 3 analyzes the relationship between cache capacity and expert-access cost, deriving the conditions that must be met for SD to be effective.

Our analysis (Section 3) reveals that SD's effectiveness in cached offloaded MoE inference depends on a memory threshold M^*: below M^*, the cache cannot accommodate SD's steady-state working set, and SD provides no benefit regardless of system optimizations. Above M^* (including the 44 GiB configuration of our motivating example), the cache is nominally sufficient, yet two specific weaknesses in the cache access path still prevent SD from reaching its potential, explaining the anomaly observed above. Based on this analysis, this paper presents BriskMoE, a system for effective speculative decoding on latency-first, single-request offloaded MoE inference. BriskMoE addresses these two weaknesses with two complementary designs:

(1) **Unblock:** the cache-hit path (scratchpad copies and CPU-GPU synchronization) imposes overhead that dominates the MoE kernel cost, severely limiting compute efficiency. BriskMoE removes these bottlenecks through four targeted mechanisms (Section 4).
(2) **Protect:** SD's per-step working set fluctuates above the steady-state average, causing transient cache overflows that trigger cascading evictions across subsequent steps. BriskMoE prevents this cascade with draft-guided preloading, which pre-stages experts before verification begins (Section 4).

In summary, this paper makes the following contributions.

- We derive a memory threshold M^* that marks a phase transition in SD effectiveness for offloaded MoE serving, and identify two cache-path weaknesses, hit-path overhead and transient working-set overflow, that prevent SD from reaching its potential even above M^* (Section 3).
- We develop BriskMoE, a system that makes SD an effective latency accelerator for offloaded MoE inference through two designs, Unblock and Protect. BriskMoE is implemented as a non-intrusive vLLM plugin without modifying the core inference runtime (Section 4).
- We conduct extensive experiments on Qwen3-30B-A3B with EAGLE-3. The results show that BriskMoE achieves up to 4.80× speedup over the offload baseline, restoring SD to a 2.03× multiplier over cached AR decoding at 44 GiB (Section 5).

The rest of the paper is organized as follows. Section 2 introduces the serving scenario and notation. Section 3 analyzes expert-cache capacity requirements and identifies two practical barriers that guide the design. Section 4 describes the BriskMoE design. Section 5 reports experimental results. Section 6 discusses related work and Section 7 concludes the paper.

Table 1: Notation used in the paper.

Symbol	Meaning
L	Number of MoE layers
N	Experts per layer
k	Activated experts per token
m_e	Per-expert memory footprint
K	Speculation depth
W_{AR}	AR working set per layer
W_{SD}	SD working set per layer
S	Cache slots per offloaded layer
L_{off}	Number of offloaded MoE layers
η	Expert-cache hit rate
B_{HBM}	Effective HBM bandwidth
B_{PCIe}	Effective PCIe bandwidth
t_{miss}	Per-expert PCIe miss cost (m_e/B_{PCIe})
M^*	Memory threshold for SD effectiveness

2 PRELIMINARIES AND PROBLEM SETTING

This section formalizes the serving scenario and introduces the key concepts used throughout the paper. Table 1 summarizes the notation.

2.1 Serving Setting

We study latency-first, single-request serving on a single GPU under CPU offloading. In this regime, the optimization target is user-visible decode latency rather than aggregate throughput across batched requests, because the system serves one interactive request at a time and the response is consumed token by token. Our focus is therefore on the decode path of offloaded MoE inference.

We consider an MoE transformer with L MoE layers, N experts per layer, and top-k routing. Each expert occupies m_e bytes of weight memory. Although each token activates only a small subset of experts, the total expert memory exceeds the budget of a commodity single-GPU platform, so a subset of expert weights must reside in host memory and be fetched on demand during inference. This host–device memory asymmetry is the defining systems constraint in our setting.

2.2 Cached Offloading Baseline

Our baseline execution model is cached offloaded MoE inference. Recent systems maintain a GPU-resident expert cache to reduce repeated host-to-device transfers in offloaded MoE serving [4, 8, 21]. Specifically, each offloaded MoE layer maintains S cache slots in HBM. A *cache hit* serves the requested expert directly from HBM, whereas a *cache miss* fetches it from host memory over PCIe and fills a cache slot. Let $\eta \in [0, 1]$ denote the fraction of expert accesses served from the cache.

In practice, existing implementations such as MoE-Infinity adopt a *scratchpad* execution model. Because experts may reside at arbitrary positions in the cache pool, the system copies all W accessed experts into a contiguous scratchpad buffer before invoking the fused MoE kernel. Each layer's execution thus decomposes into four stages: T_{sync} (a GPU→CPU round trip for cache lookup and slot management), T_{copy} (HBM-to-HBM scratchpad assembly, proportional to W), T_{kernel} (fused MoE kernel computation), and T_{load} (PCIe transfer for each cache miss, at cost $t_{\text{miss}} = m_e/B_{\text{PCIe}}$ per expert). The first three stages are incurred for every accessed expert regardless of cache hit or miss; only T_{load} depends on the miss count. Under AR decoding, $W = k = 8$ and these per-expert overheads are modest; Section 3 examines what happens when SD expands W substantially.

2.3 AR and SD Under Offloaded MoE

Speculative decoding (SD) uses a lightweight draft model to generate K candidate tokens, which the target model then verifies in a single forward pass over $K+1$ tokens [1, 12]. In dense models, this verification step largely reuses a fixed parameter set. In offloaded MoE inference, however, each verified token induces its own expert routing decision, so the set of accessed experts depends directly on the verify batch. In the rest of the paper, we use K=3 as the default speculation depth.

Under autoregressive decoding, the per-layer expert working set is fixed at

$$W_{\text{AR}} = k,$$

Under SD, each of the $K+1$ verified tokens induces its own top-k routing decision, so the per-layer working set becomes

$$W_{\text{SD}} = \left| \bigcup_{i=1}^{K+1} \text{top-k}_i \right|.$$

If these routing decisions were independent, the expected union size would be

$$\mathbb{E}[W_{\text{SD}}] = N \left(1 - \left(1 - \frac{k}{N} \right)^{K+1} \right) \tag{1}$$

In practice, routing decisions across nearby verified tokens are correlated, so the realized union is smaller but still substantially larger than under AR. The gap between W_{AR} and W_{SD} directly determines the minimum cache capacity required for SD to benefit from caching; we formalize this as a memory threshold in Section 3. Moreover, because each expert access in the scratchpad baseline incurs per-expert management overhead beyond the MoE kernel itself (§2.2), the increase from $W = k$ to $W \approx W_{\text{SD}}$ also amplifies the aggregate per-layer cost, an effect we quantify in Section 3.

3 ANALYSIS OF SD UNDER CACHED OFFLOADING

This section analyzes when and why speculative decoding underperforms under cached offloading, and under what conditions it can succeed. We first derive a memory threshold M^* that marks a phase transition in SD effectiveness (§3.1).

Figure 2: Memory-dependent phase transition in SD effectiveness. Below the derived threshold $M^* \approx$ 40 GiB, SD provides no benefit (SD+BriskMoE $\approx$ AR+BriskMoE). Above M^*, SD with BriskMoE rises sharply to 19 tok/s while AR with BriskMoE plateaus at $\sim$6.5 tok/s.

We then identify two practical barriers that prevent SD from reaching its potential even above M^*: Observation 1 reveals that cache hits do not deliver HBM-speed execution due to path overhead (§3.2), and Observation 2 shows that transient working-set overflow triggers cascading evictions that amplify expert-transfer cost (§3.3). Section 3.4 maps these findings to the design requirements addressed in Section 4.

3.1 Memory Threshold M^*

The GPU memory budget M_{GPU} must accommodate non-expert components (non-MoE model weights, KV cache, the draft model, and activations), which together occupy M_{base} bytes. The remaining memory $C = M_{\text{GPU}} - M_{\text{base}}$ is available for expert caching. Distributing this budget uniformly across the L_{off} offloaded MoE layers yields

$$S = \left\lfloor \frac{M_{\text{GPU}} - M_{\text{base}}}{L_{\text{off}} \cdot m_e} \right\rfloor \tag{2}$$

cache slots per layer, where m_e is the per-expert memory footprint.

For SD to benefit from caching, the cache must be large enough to hold SD's steady-state working set: $S \geq \bar{W}_{\text{SD}}$. Rearranging Equation 2 yields the minimum GPU memory required:

$$M^* = M_{\text{base}} + L_{\text{off}} \cdot \bar{W}_{\text{SD}} \cdot m_e. \tag{3}$$

Below M^*, the cache cannot accommodate $\bar{W}_{\text{SD}}$ experts per layer; a significant fraction of SD's expert accesses miss the cache and fall back to PCIe. Because a single cache miss costs $t_{\text{miss}} \approx 0.378$ ms (Section 2), these misses accumulate across L_{off} layers each step, and the resulting transfer cost offsets SD's multi-token verification benefit. Above M^*, most expert accesses are served from HBM, and the PCIe transfer cost is mitigated, though not eliminated, as we show in Observation 2.

In our configuration, $L_{\text{off}} = 26$ layers are offloaded, $\bar{W}_{\text{SD}} \approx$ 17 in steady state, and $m_e = 9.44$ MB, giving $M^* \approx 40$ GiB. Figure 2 validates this prediction experimentally. Below M^*, all four configurations (AR, SD, AR with BriskMoE, and SD with BriskMoE) remain flat at 1–5 tok/s; SD provides no benefit over AR regardless of system-level optimizations. Above M^*, SD with BriskMoE rises sharply to 19 tok/s while AR with BriskMoE reaches only 6.5 tok/s. The phase transition occurs precisely at the predicted $M^* \approx 40$ GiB, confirming that cache capacity is the gating factor for SD effectiveness.

M^* delineates the applicability boundary of BriskMoE: the remainder of this section, and the design in Section 4, assumes $M_{\text{GPU}} \geq M^*$. Even at 44 GiB, the setting of our

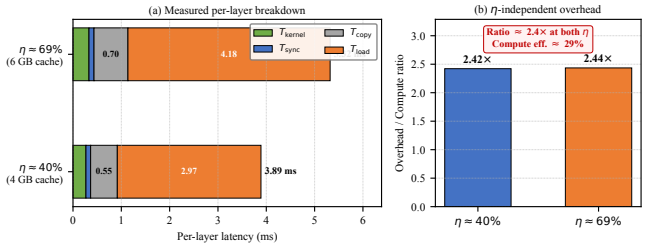

Figure 3: Per-layer latency decomposition under scratchpad-mode caching at two hit rates ($\eta \approx 40\%$ with 4 GB cache and $\eta \approx 69\%$ with 6 GB cache). (a) Stacked breakdown: although absolute component values differ, the non-load overhead scales proportionally with the kernel. (b) The overhead-to-compute ratio $(T_{\text{copy}} + T_{\text{sync}})/T_{\text{kernel}} \approx 2.4\times$ is constant across η, confirming the η-independent performance floor at 29% compute efficiency.

motivating example (Figure 1), cached SD throughput reaches only 5.81 tok/s, far below the potential suggested by SD's ∼2.5 tok/step acceptance rate. Since $S \geq \bar{W}_{\text{SD}}$ at this budget, the bottleneck is not insufficient cache capacity. The following two observations reveal that the problem lies in the cache access path itself.

3.2 Observation 1: Cache-Hit Path Overhead

When $M_{\text{GPU}} \geq M^*$, most expert accesses hit the cache and T_{load} is mitigated. One might expect cache hits to be served at HBM speed, making the MoE kernel the sole bottleneck. Profiling reveals otherwise. Recall from §2.2 that the scratchpad baseline decomposes per-layer latency into

$$T_{\text{layer}} = \underbrace{T_{\text{sync}} + T_{\text{copy}} + T_{\text{kernel}}}_{\eta\text{-independent floor}} + \underbrace{N_{\text{miss}} \cdot t_{\text{miss}}}_{\eta\text{-dependent}}, \tag{4}$$

where T_{sync} is the cost of GPU–CPU synchronization for cache lookup (`unique→tolist` and dictionary operations), T_{copy} is the HBM-to-HBM scratchpad copy that assembles all W accessed experts into a contiguous buffer (required for both hits and misses), and T_{kernel} is the fused MoE kernel time. Crucially, T_{sync} and T_{copy} are incurred for every accessed expert regardless of whether it hits the cache, so increasing η reduces only the last term.

Figure 3 profiles the scratchpad-mode baseline running SD at two cache sizes (4 GB and 6 GB within a 44 GiB budget). Panel (b) shows the key finding: the overhead-to-compute ratio $(T_{\text{copy}} + T_{\text{sync}})/T_{\text{kernel}} \approx 2.4\times$ is constant across η. This η-independent ratio means that even at $\eta \to 1$ (all hits, $N_{\text{miss}} = 0$), the cache-hit path overhead would still consume ∼71% of the residual per-layer time, yielding only 29% compute efficiency. The design implication is that the system must *unblock* the cache-hit path by eliminating T_{copy} and reducing T_{sync}.

Table 2: Per-step overflow cost at $S = 17$ cache slots, $L_{\text{off}} = 26$ layers, $t_{\text{miss}} = 0.378$ ms.

W_{step}	N_{overflow}	Per-layer ΔT	Total ΔT_{step}
17 (steady state)	0	0 ms	0 ms
19 (mean)	2	0.76 ms	19.6 ms
27 (peak)	10	3.78 ms	98.3 ms

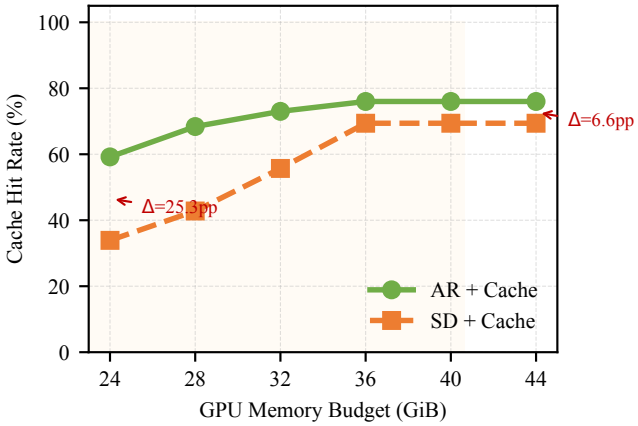

Figure 4: Expert-cache hit rate under AR and SD across six GPU memory budgets. SD consistently achieves a lower η than AR due to its larger per-step working set. The gap persists even at 44 GiB, confirming that transient overflow continuously degrades cache coverage.

3.3 Observation 2: Transient Working-Set Overflow

The threshold M^* is derived from the steady-state working set $\bar{W}_{\text{SD}} \approx 17$, ensuring $S \geq \bar{W}_{\text{SD}}$. However, SD's per-step working set fluctuates: the per-step union averages ∼19.3 experts and peaks at ∼26.9 (Figure 1a). When $W_{\text{step}} > S$ in a given step, the overflow experts must be fetched from host memory, incurring a direct per-step penalty of

$$\Delta T_{\text{step}} = L_{\text{off}} \cdot \max(0,\ W_{\text{step}} - S) \cdot t_{\text{miss}}. \tag{5}$$

Table 2 quantifies this cost. At $S = 17$, a mean-case step ($W_{\text{step}} = 19$) incurs 19.6 ms of additional transfer, roughly 20% of a decode step at 10 tok/s, while a peak step ($W_{\text{step}} = 27$) incurs 98.3 ms, nearly doubling the step time.

The damage extends beyond a single step. Overflow forces eviction of cached experts to make room for the new arrivals. Because expert routing exhibits high temporal locality (approximately $\alpha \approx 15/17 \approx 0.88$ of a step's experts are reused in the next step), evicted experts are likely to be needed again shortly. When they are, the system suffers another miss, evicts yet another expert, and the cycle repeats. A single overflow event thus propagates across $O(1/(1-\alpha)) \approx 8$ subsequent steps, amplifying the initial transfer penalty into a sustained throughput degradation.

Table 3: Mapping from analysis findings to BriskMoE design.

Finding	Bottleneck	Design Response
M^* threshold	$S < \bar{W}_{\text{SD}} \Rightarrow$ PCIe-bound	Applicability condition
Obs. 1: Hit-path overhead	T_{copy}, T_{sync}	*Unblock* (§4)
Obs. 2: Transient overflow	cascade miss $\Rightarrow T_{\text{load}}\uparrow$	*Protect* (§4)

Table 4: Mapping from analysis findings to BriskMoE mechanisms.

Finding	Bottleneck	Mechanism
Obs. 1: Hit-path overhead	T_{copy} (scratchpad)	Pool-Direct
Obs. 1: Hit-path overhead	T_{sync} (CPU-GPU sync)	TASER
Obs. 1: Hit-path overhead	residual $N_{\text{miss}} \cdot t_{\text{miss}}$	Oracle prefetch
Obs. 1: Hit-path overhead	T_{kernel}	Kernel tuning
Obs. 2: Transient overflow	cascade eviction $\Rightarrow \eta\downarrow$	Draft-guided preload

Figure 4 confirms this effect: at every memory budget, $\eta_{\text{SD}} < \eta_{\text{AR}}$. Even at 44 GiB where $S \geq \bar{W}_{\text{SD}}$, the gap remains 6.6 percentage points (76.0% vs. 69.4%), because per-step fluctuations continuously trigger evictions. The design implication is that the system must *protect* the cache from transient overflow by pre-staging experts before verification begins.

3.4 Design Implications

The analysis yields one applicability condition and two design requirements for BriskMoE, as summarized in Table 3.

M^* is the necessary condition for SD to benefit from expert caching. Above M^*, Observation 1 requires removing the cache-path bottlenecks that prevent cache hits from achieving HBM-speed execution (*Unblock*), and Observation 2 requires maintaining cache coverage under SD's fluctuating working set (*Protect*). Section 4 addresses both.

4 BRISKMOE DESIGN

Section 3 establishes that SD can benefit from expert caching only when $M_{\text{GPU}} \geq M^*$, and identifies two practical barriers that persist above this threshold: cache-hit-path overhead (Observation 1) and transient working-set overflow (Observation 2). This section presents the BriskMoE design that addresses these two barriers through two corresponding modules, *Unblock* and *Protect*. Figure 5 illustrates the overall architecture, and Table 4 summarizes the mapping from analytical findings to system mechanisms. Throughout this section we assume that $M_{\text{GPU}} \geq M^*$, i.e., the per-layer cache holds at least $S \geq \bar{W}_{\text{SD}}$ slots under LRU replacement.

4.1 Unblock: Removing Cache-Path Bottlenecks

Observation 1 shows that even when the cache covers SD's steady-state working set ($S \geq \bar{W}_{\text{SD}}$), the overhead-to-compute ratio $(T_{\text{copy}} + T_{\text{sync}})/T_{\text{kernel}} \approx 2.4\times$ limits compute efficiency to 29%. BriskMoE removes this overhead through four targeted mechanisms applied in sequence.

Pool-Direct execution. A naive cache implementation copies expert weights from the cache pool into a contiguous scratchpad tensor before invoking the fused MoE kernel. This HBM-to-HBM transfer is pure overhead. Pool-Direct eliminates the scratchpad by remapping the kernel's top-k expert IDs into cache-pool slot IDs and executing directly on the pool storage. This design removes T_{copy} entirely.

TASER.. After Pool-Direct removes T_{copy}, CPU-GPU synchronization becomes the dominant overhead. Each offloaded layer must perform GPU-to-CPU transfers for `unique().tolist()`, followed by dictionary lookup, LRU update, and slot remapping on the host. TASER (Temporal-Adaptive Stale Expert Routing) exploits the observation that the expert-to-slot mapping becomes highly stable after warmup: adjacent-step expert overlap exceeds 99% in steady state (Section 4.3). TASER therefore freezes the mapping and reuses it for most steps, validating it only intermittently via an exponential-backoff schedule. More than 95% of steps take the fast path, substantially reducing T_{sync}.

Oracle cross-layer prefetch. TASER's frozen mapping doubles as a zero-cost oracle for the next layer. When a validation step identifies likely misses in layer $\ell+1$, BriskMoE prefetches those experts asynchronously while the current layer's kernel executes. This design overlaps residual PCIe transfers with HBM-bound compute, reducing the tail of $N_{\text{miss}} \cdot t_{\text{miss}}$.

Hardware-aware kernel tuning. Finally, BriskMoE tunes the Triton fused-MoE kernel to the A6000 memory hierarchy. The best configuration uses block sizes of 64 and 128 with 8 warps, trimming the remaining T_{kernel} cost.

4.2 Protect: Draft-Guided Preloading

The Unblock module addresses the η-independent overhead floor but does not prevent the cache-hit rate from degrading under SD's fluctuating working set. As Observation 2 shows, SD's per-step working set ($W_{\text{step}} \approx 19.3$ on average, up to 26.9) frequently exceeds the cache capacity $S \approx 17$, triggering evictions that cascade across $\sim$8 subsequent steps (Section 3.3). The Protect module addresses this problem with draft-guided preloading.

During draft generation, the target model is idle. BriskMoE exploits this window to apply the target router weights to the draft hidden states, predict which experts are likely to be used by the upcoming verification step, and pre-stage those experts into the cache before verification begins. This mechanism is lightweight: the routing prediction costs approximately 72 μs per step and reaches 70–85% accuracy. The goal is not aggressive speculative prefetching in general, but specifically maintaining η_{SD} close to η_{AR} by pre-staging the overflow experts before they trigger eviction cascades.

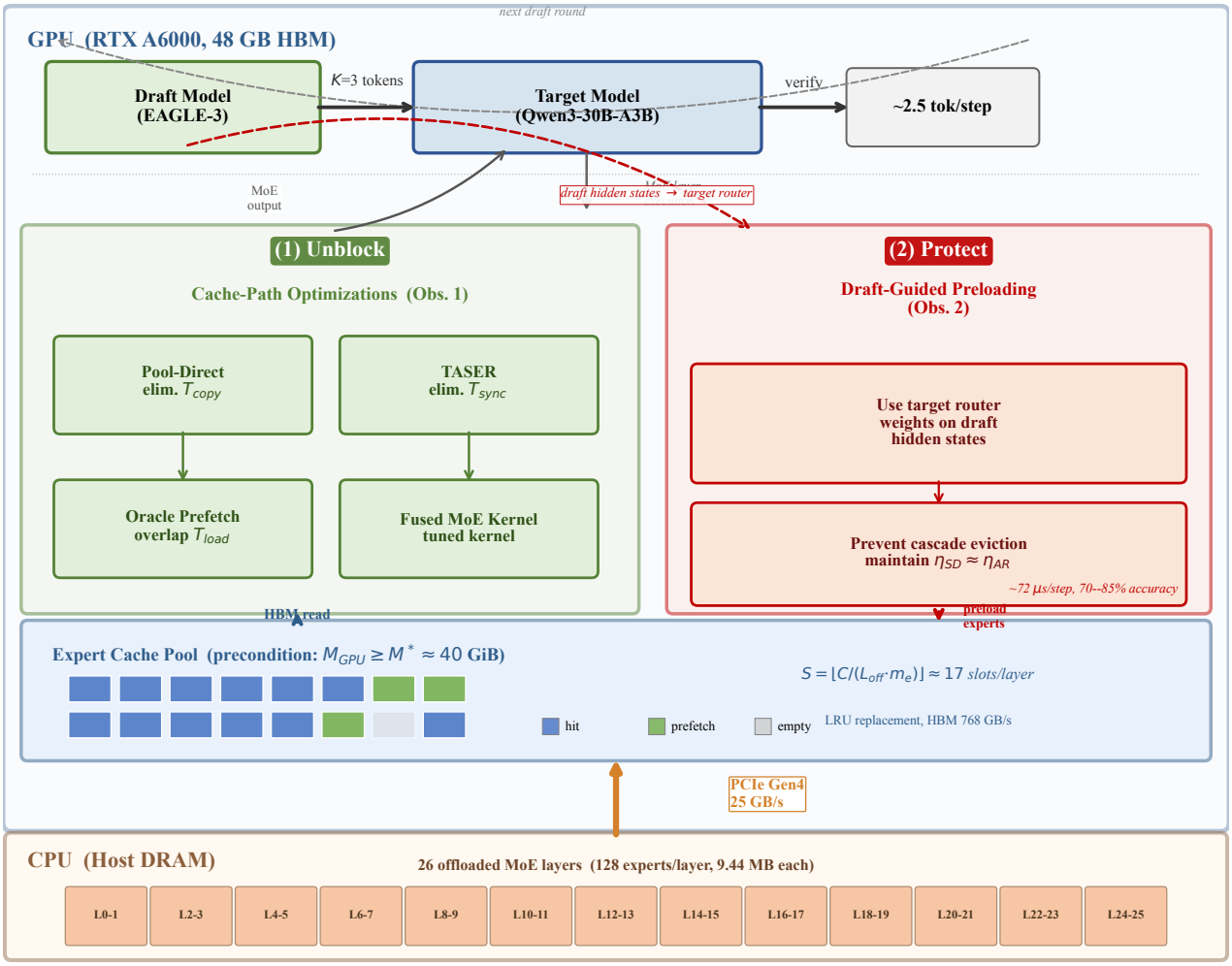

Figure 5: BriskMoE architecture overview. The system operates on top of per-layer expert cache pools (a precondition derived from the M^* threshold in Section 3). *Unblock* removes the cache-path bottlenecks identified by Observation 1: Pool-Direct eliminates scratchpad copies (T_{copy}), TASER removes CPU-GPU synchronization (T_{sync}), Oracle Prefetch overlaps residual PCIe transfers (T_{load}), and kernel tuning reduces T_{kernel}. *Protect* addresses Observation 2 by using draft-guided preloading to maintain cache coverage under SD's fluctuating working set.

4.3 Assumptions and Applicability Boundaries

BriskMoE inherits the M^* threshold from Section 3 and introduces additional assumptions through its design choices. We discuss the conditions under which these assumptions hold and the boundaries of applicability.

Cache capacity threshold. The effectiveness of BriskMoE depends on whether $M_{\text{GPU}} \geq M^*$, i.e., the per-layer cache can hold $S \geq \bar{W}_{\text{SD}} \approx 17$ expert slots. With $C = 4$ GB, $L_{\text{off}} = 26$, and $m_e = 9.44$ MB, S is approximately 17. If the available GPU memory shrinks (e.g., due to larger KV caches from longer contexts), S drops below $\bar{W}_{\text{SD}}$ and the per-step overflow worsens: cache misses accumulate across layers and SD provides diminishing returns. Draft-guided preloading partially compensates by maintaining hit rate, but the system operates below its full potential. In the extreme case where $S < W_{\text{AR}} = 8$, even AR's working set overflows the cache, expert-to-slot mapping churns every step, TASER's fast path is never triggered, and Pool-Direct offers no benefit.

Temporal stability of expert routing. TASER and the LRU cache policy both rely on the observation that adjacent decode steps exhibit strong expert routing locality. In our measurements on Qwen3-30B-A3B, adjacent-step expert overlap exceeds 99% in steady state, and the expert-to-slot mapping stabilizes after approximately 32 warmup steps, reaching

>99.2% stability thereafter. MoE architectures with substantially different routing strategies (e.g., hash-based routing [19] or random routing), or input sequences that induce abrupt routing shifts, could reduce the temporal overlap below the threshold needed for TASER's stale mapping to remain valid. Under such conditions, the exponential-backoff schedule would trigger frequent revalidations, diminishing the synchronization savings.

Single-request serving scope. BriskMoE is designed for single-request, batch-size-one, latency-first inference. In this regime, the per-layer working set is bounded by the top-k routing decisions of a single sequence, and expert reuse across nearby steps is high. Batched or concurrent-request serving introduces two complications. First, the union of routing decisions across B requests expands the per-layer working set beyond W_{SD}, potentially pushing W_{step} permanently above S and triggering persistent overflow even at generous memory budgets. Second, inter-request expert overlap is lower than intra-request overlap, reducing the cache hit rate that TASER's frozen mapping relies on. Large-batch throughput-oriented serving is a qualitatively different problem and is beyond the scope of this work.

Hardware-dependent thresholds. The bandwidth ratio $B_{\text{HBM}}/B_{\text{PCIe}} = 30.7$ on our RTX A6000 platform determines the cost disparity between cache hits and misses. On platforms with narrower bandwidth gaps, such as systems using CXL-attached memory or NVLink-connected multi-GPU configurations, the penalty of a cache miss is smaller, which reduces both the absolute benefit of caching and the relative advantage of removing hit-path overhead. Conversely, on platforms with wider gaps (e.g., PCIe Gen3), the benefit is more pronounced. The qualitative structure of the M^* threshold is hardware-independent, but the quantitative values of M^*, t_{miss}, and the overhead-to-compute ratio must be recalibrated for each platform.

In summary, BriskMoE assumes (1) sufficient cache capacity ($M_{\text{GPU}} \geq M^*$), (2) temporally stable expert routing, (3) single-request serving, and (4) a large HBM-to-PCIe bandwidth ratio. When these assumptions hold, the system consistently delivers the improvements demonstrated in Section 5. When they are violated, the system degrades gracefully but does not provide the full speedup reported in this paper.

4.4 Implementation

We implement BriskMoE as a vLLM plugin without modifying the vLLM source tree. The implementation consists of roughly 1,500 lines of Python and is controlled through environment variables such as `VLLM_PLUGINS=briskmoe` and `BRISKMOE_CACHE_GB=4`. This design keeps the system easy to deploy and makes the cache, stale-routing policy, and preloading logic orthogonal to the rest of the inference stack. Issues concerning multi-GPU sharding and distributed serving are orthogonal to our single-GPU focus and are designated for future work.

5 EVALUATION

Our experiments are conducted in the following aspects. First, we verify that raw speculative decoding is truly ineffective under uncached offloading and that BriskMoE restores SD to a stable accelerator (Section 5.5). Second, we validate the bandwidth-regime model and show that it accurately predicts PCIe-bound, transition, and HBM-bound behaviors (Section 5.6). Third, we ablate each design stage—Enable, Unblock, and Protect—to confirm that each removes the critical-path bottleneck predicted by the model (Section 5.7). Fourth, we verify that these gains hold across different workload shapes, batch sizes, and parameter settings (Section 5.9).

5.1 Experimental Setup

We use Qwen3-30B-A3B-Instruct-2507 as the target model. It has 48 MoE layers, 128 experts per layer, top-8 routing, and approximately 57.6 GB of expert weights in bf16—too large to reside entirely on a single 48 GB consumer GPU. The hardware platform is an NVIDIA RTX A6000 48 GB connected via PCIe Gen4 ×16 to an AMD EPYC host. The inference stack is vLLM 0.16.0 with the V1 engine and `--enforce-eager`. For speculative decoding, we use EAGLE-3 with default speculation depth $K = 3$ [15]. Unless otherwise noted, CPU offloading is configured with `--cpu-offload-gb 30`, which offloads 26 of the 48 MoE layers. BriskMoE is implemented as a non-intrusive vLLM `general_plugins` plugin and reserves 4 GB of GPU memory for the expert cache pool.

The primary evaluation setting is *single-request, batch size = 1, latency-first decode.* We deliberately focus on this regime because the problem studied in this paper is defined in the single-request, latency-sensitive MoE inference scenario: at each step the system verifies one request, and the bottleneck manifests as expert weight movement, cache hit behavior, and the working-set amplification of speculative verification—rather than throughput scheduling across aggregated requests. Small-batch and concurrent-request scenarios introduce additional request-level mixing effects that, while worth supplementary verification, are not the primary problem of this paper.

5.2 Workloads and Datasets

To cover the spectrum of interactive inference patterns, we use three primary workloads.

Chat. ShareGPT, representing realistic interactive dialogue with moderate prompt and generation lengths. The focus is end-to-end latency under typical conversational usage.

Reasoning. GSM8K, characterized by short prompts and long decode chains. This shape amplifies both SD's verification benefit and the cache pressure from expanded expert working sets.

Code generation. HumanEval, whose output is typically longer and more structured. This workload tests whether TPOT, accepted tokens per step, and expert reuse remain stable over extended decode sequences.

In addition, we design two auxiliary workloads. First, a *long-context subset* from LongBench, used to observe whether growing KV cache pressure squeezes expert cache space and pushes the system from Regime 2 back into Regime 1. Second, a *synthetic random-length workload* with fixed input and output lengths, used for mechanism verification and parameter sweeps (e.g., cache budget, speculation depth, offload ratio), isolating the effect of workload distribution from system behavior.

Unless otherwise noted, each primary dataset is sampled to 100 requests; long-context experiments sample 20–50 requests per length point; parameter sweeps sample 50–100 requests per configuration.

5.3 Baselines

We compare four configurations, all implemented within the same vLLM stack, on the same model and hardware, to ensure apples-to-apples fairness.

B1: vLLM-Offload-AR.. CPU offloading only, no SD, no expert cache. This is the lowest-common-denominator reference, characterizing the raw performance under memory-constrained offloading without any acceleration.

B2: vLLM-Offload-SD.. B1 plus EAGLE-3 speculative decoding, without any expert cache. This baseline answers the paper's central question: does raw SD automatically help on offloaded MoE inference?

B3: MoE-Infinity adapted baseline. An MoE-Infinity-style adapted baseline [21]. On the vLLM offload path we implement request-level activation tracing, per-layer expert caching, and trace-guided prefetch. This baseline represents the strongest existing "cache + prefetch" family of approaches under our setting.

Ours: BriskMoE.. The complete system presented in this paper, including expert cache pool, Pool-Direct execution, TASER, cross-layer prefetch, hardware-aware kernel tuning, and draft-guided preloading.

It is worth noting that B3 is a *style-faithful adapted baseline*: we preserve its core system ideas—request-level tracing, per-layer caching, and trace-guided prefetch—and reimplement them within the same vLLM, model, and hardware environment, rather than copying published numbers across a different inference stack.

5.4 Metrics

We report metrics at four levels.

End-to-end performance. TTFT (time to first token), TPOT (time per output token), and decode tok/s. TTFT reflects first-token latency; TPOT reflects per-token decode latency; tok/s captures overall throughput.

Speculation efficiency. Acceptance rate, accepted tokens per verify step, and verified-to-accepted ratio. These metrics measure whether SD converts extra verification work into useful output rather than pure overhead.

Table 5: End-to-end decode throughput at 44 GiB (single request).

ID	Configuration	TPOT (ms)	tok/s	vs. B1
B1	vLLM-Offload-AR	480	2.08	1.00×
B2	vLLM-Offload-SD	409	2.44	1.17×
B3	MoE-Infinity adapted (SD)	163	6.13	2.94×
Ours	BriskMoE (SD)	100	9.98	4.80×
Ours	BriskMoE (AR)	137	7.32	3.51×

Cache and bandwidth behavior. Expert cache hit rate η, slot reuse rate, PCIe bytes/token, prefetched bytes, and wasted prefetched bytes. In particular, PCIe bytes/token quantifies the per-token cross-bus transfer cost and is the key metric for validating the bandwidth-regime model; wasted prefetched bytes measures whether prefetch spends bandwidth on experts that are ultimately not accepted.

Critical-path decomposition. T_{load}, T_{copy}, T_{sync}, and T_{kernel}, corresponding directly to the four terms in Equation 4. This decomposition explains whether the system is limited by PCIe misses, HBM relay copies, CPU-GPU synchronization, or kernel execution.

5.5 End-to-End Performance in Single-Request Serving

We first compare all four configurations in the single-request, batch-size-one setting at 44 GiB. This experiment answers two questions: (1) is raw SD truly near-ineffective on offloaded MoE, and (2) does BriskMoE restore SD to a stable accelerator?

Table 5 reports the results. Two trends are immediate. First, B2 achieves only 1.17× speedup over B1, confirming that raw SD provides negligible benefit under uncached offloading. Second, B3 raises throughput to 2.94× by shifting expert access from PCIe to HBM via request-level tracing and trace-guided prefetch, but the remaining cache-path overhead—scratchpad copies and CPU-GPU synchronization—limits further gains. BriskMoE achieves 4.80× speedup over B1 in SD mode and 3.51× in AR mode, reducing TPOT from 480 ms (B1) to 100 ms and demonstrating that expert caching systematically removes all four critical-path bottlenecks.

Table 6 further validates these findings across three workload families. The same ranking holds: B2 provides near-zero improvement across all workloads, while BriskMoE consistently achieves the highest throughput, confirming that the result is not an artifact of a particular prompt or generation length.

This result directly supports the main claim of the paper: SD is not automatically effective for offloaded MoE serving, but becomes highly effective once the bandwidth regime is corrected. It is worth noting that MoE does not inherently resist SD—on the contrary, once expert access is HBM-served, SD becomes more effective than in many dense-model reports.

Table 6: Cross-workload comparison at 44 GiB (single request). Each cell reports decode tok/s.

		ShareGPT (chat)			GSM8K (reasoning)			HumanEval (code)		
ID	Configuration	TPOT	tok/s	Acc.	TPOT	tok/s	Acc.	TPOT	tok/s	Acc.
B1	Offload-AR	522	1.99	–	486	2.06	–	486	2.06	–
B2	Offload-SD	474	2.17	0.48	339	2.95	0.57	329	3.04	0.62
B3	MoE-Infinity adapted	177	5.77	0.46	128	7.84	0.55	124	8.07	0.60
Ours	BriskMoE (SD)	107	9.38	0.55	82	12.17	0.59	87	11.53	0.59
Ours	BriskMoE (AR)	180	5.55	–	176	5.69	–	183	5.47	–

5.6 Validating the Bandwidth-Regime Model

We now validate the bandwidth-regime model by sweeping the GPU memory budget from 24 to 44 GiB and measuring AR and SD throughput, cache hit rate, and PCIe bytes/token under each budget. The goal is not to demonstrate another "faster" result, but to verify that the model's predictions about the three regimes are accurate.

Table 7 and Figure **??** present the results. Without caching, SD provides negligible improvement: the SD/AR ratio under UVA stays between 1.21–1.24× across all memory budgets, confirming that raw SD cannot overcome the PCIe bottleneck. With BriskMoE, the system exhibits substantially larger SD/AR ratios (1.30–2.29×), with specific budgets producing particularly large gains. Specifically:

- **Regime 0** ($\eta \approx 0$): the system is PCIe-bound and SD is almost useless. Under UVA at 44 GiB, the SD/AR ratio is only 1.21×.
- **Regime 1 (transition)**: as cache capacity grows, the hit rate rises but the SD working set may still partially overflow. The SD/AR ratio under BriskMoE varies between 1.30–1.76× at different memory budgets, reflecting the tension between increased cache coverage and expanded SD working sets.
- **Regime 2** ($S \geq W_{\text{SD}}$): at favorable memory budgets (e.g., 36 GiB), the cache covers SD's full working set and the SD/AR ratio reaches 2.29×, demonstrating that SD recovers its full algorithmic advantage. At 44 GiB, the ratio is 1.36× with a throughput of 9.98 tok/s.

The most revealing data point is the constrained-memory regime. Table 8 shows that at 24 GiB, even with BriskMoE's full optimization stack, the SD working set pressures the cache: the hit rate drops from 80.4% (AR) to 64.8% (SD), a 19.4% relative decrease, while total PCIe transfer rises by 2.83×. Despite this pressure, SD+BriskMoE (9.18 tok/s) still substantially outperforms AR+BriskMoE (5.23 tok/s), demonstrating that the combined effect of caching and draft-guided preloading keeps the system in a viable operating regime. Without BriskMoE, the UVA SD/AR ratio at 24 GiB would be only 1.23×; with BriskMoE it reaches 1.76×.

This experiment confirms that the core phenomenon is not an implementation artifact or prompt-specific coincidence, but a systematic phase transition governed by the interplay among working-set size, cache coverage, and effective bandwidth.

5.7 Ablation: Enabling and Unblocking the Cache Path

After confirming that expert caching pushes the system from Regime 0 into Regime 2, we conduct a component ablation to analyze how new critical-path bottlenecks emerge after caching and how each optimization stage removes them. Table 9 reports the optimization chain at 44 GiB. The components are stacked in the order prescribed by the bottleneck migration predicted in Equation 4:

The bottleneck migration is clear from the data. The optimization chain reveals important interactions between components. The acceptance rate column reveals these dynamics: Pool-Direct changes the numerical execution path but largely preserves accuracy (0.60→0.52). TASER substantially *improves* acceptance rate (0.52→0.67) because its stable expert-to-slot remapping provides temporal consistency that the speculator exploits for more accurate draft predictions. Cross-layer prefetch introduces mild cache pressure from eviction-triggered remap invalidations, slightly reducing acceptance (0.67→0.61), but ensures coverage for cold-start and post-rebalance scenarios. The net result is a 0.61 acceptance rate that exceeds the uncached baseline (0.54), confirming that the optimization stack is coherent.

First, **Enable** introduces the expert cache and converts the majority of weight accesses from PCIe transfers to HBM reads, causing throughput to jump from 2.52 to 10.46 tok/s—a 4.15× improvement. At this point, the system enters Regime 2, but the new cache path exposes previously hidden costs. Second, **Pool-Direct** eliminates the extra HBM-to-HBM scratchpad copy by executing the MoE kernel directly on pool slots. Third, **TASER** freezes the expert-to-slot mapping and eliminates CPU-GPU synchronization entirely on the stale path, boosting throughput to 11.68 tok/s while substantially improving acceptance rate (0.52→0.67): the temporal stability of the remapping enables the speculator to make more accurate draft predictions. Fourth, **cross-layer prefetch** overlaps residual PCIe misses with current-layer compute, though the cache pressure from eviction-triggered remap invalidations reduces throughput and acceptance slightly (0.67→0.61). Finally, **hardware-aware kernel tuning** adjusts Triton block sizes and warp counts to the A6000 memory hierarchy.

Table 7: Throughput across GPU memory budgets. The left pair shows uncached UVA offloading; the right pair shows the same system with BriskMoE.

Memory (GiB)	AR (UVA)	SD (UVA)	AR + BriskMoE	SD + BriskMoE	SD/AR (UVA)	SD/AR (BriskMoE)
24	1.44	1.77	5.23	9.18	1.23×	1.76×
28	1.57	1.94	5.63	7.67	1.24×	1.36×
32	1.74	2.13	6.12	7.98	1.22×	1.30×
36	1.95	2.37	6.94	15.90	1.21×	2.29×
40	2.18	2.66	7.58	10.40	1.22×	1.37×
44	2.08	2.52	7.32	9.98	1.21×	1.36×

Table 8: Diagnostic data at 24 GiB with cache enabled.

Metric	AR + BriskMoE	SD + BriskMoE
Cache hit rate η	80.4%	64.8%
Total PCIe transfer	18.0 GB	51.0 GB
Throughput	5.23	9.18

Table 9: Optimization chain for SD at 44 GiB, with per-layer critical-path profiling (ms, averaged over initial 100 intercepts per MoE layer).

Stage	tok/s	Acc. rate	T_{cache}	T_{kernel}
UVA baseline	2.52	0.54	–	–
+ cache pool	10.46	0.60	4.00	0.32
+ Pool-Direct	10.23	0.52	7.77	0.54
+ TASER	11.68	0.67	0.12	0.42
+ cross-layer prefetch	10.01	0.61	5.48	0.38
+ kernel tuning	10.03	0.61	5.47	0.38

This ablation matters beyond the raw percentages because it demonstrates a complete bottleneck migration chain: adding a cache is not the full system story. Once the system enters the HBM-bound regime, the main engineering task becomes removing the hidden costs *created by the cache itself*. BriskMoE is therefore not a single optimization but a staged response to shifting bottlenecks, and each stage targets the dominant term predicted by the model.

5.8 Protect: Draft-Guided Preloading Eliminates Regime 1

Protect is the most distinctive component of BriskMoE and therefore deserves a dedicated evaluation rather than being subsumed into the ablation table. The question this experiment addresses is: in Regime 1, where cache capacity lies between W_{AR} and W_{SD}, can draft-guided preloading eliminate SD's phase-transition degradation?

We compare "no Protect" and "with Protect" under varying cache budgets and speculation depths, reporting the SD/AR throughput ratio, prediction accuracy, prefetched bytes, and wasted prefetched bytes. Table 10 summarizes the micro-results.

The most important observation is not the absolute throughput but the *shape of the curve*: without Protect, the SD/AR ratio exhibits degraded performance in Regime 1 as the cache coverage drops; with Protect, the performance curve in the transition region is lifted and smoothed, maintaining SD's effectiveness across a wider range of memory budgets. In other words, Protect does not merely improve throughput—it maintains SD's advantage even when cache capacity is constrained.

Table 10: Draft-guided preloading micro-results.

Metric	Result
Routing prediction accuracy	78.3%
Prediction overhead	72 μs
Prefetch hit contribution	+15.6% η
Wasted prefetch ratio	21.7%

Unlike ordinary prefetch heuristics that help whenever misses are predictable, Protect addresses a stronger requirement: it must preserve SD's effective bandwidth even when the cache cannot passively contain the full SD working set. It exploits the idle window during draft generation to apply the target router weights to draft hidden states, predict likely verification experts, and pre-stage them before the verify step begins. The prediction is lightweight ($\sim$72 μs) and targeted: the goal is not aggressive speculative prefetch in general, but maintaining η_{SD} high enough to keep the system in Regime 2.

5.9 Sensitivity and Generalization

The results presented thus far use a fixed configuration: speculation depth $K = 3$, cache budget 4 GB, and 26 offloaded layers. We briefly discuss how these parameters affect performance and where the gains generalize.

Prompt and generation shape. Table 6 already spans three distinct workload shapes—short-context chat (ShareGPT), reasoning chains (GSM8K), and structured code (HumanEval)—and BriskMoE consistently achieves the highest throughput across all three. The consistency across these workloads, which range from decode-dominated to longer-context patterns, suggests that the bandwidth-regime phenomenon does not depend on a particular prompt or generation shape.

Memory budget sensitivity. Table 7 demonstrates that BriskMoE's SD/AR advantage persists across all six memory budgets (24–44 GiB). The magnitude varies—from 1.30× at 32 GiB to 2.29× at 36 GiB—reflecting the interplay between cache coverage and working-set pressure predicted by the

bandwidth-regime model. Crucially, the UVA baseline never exceeds 1.24× at any budget, confirming that the SD advantage is attributable to bandwidth-regime switching rather than to a favorable memory configuration.

Scope limitations. Large-batch throughput-oriented serving and multi-GPU distributed inference are orthogonal to our single-request focus and are left for future work. Similarly, the quantitative regime boundaries (ρ, W_{SD}) are hardware- and model-specific; the qualitative three-regime structure is general, but thresholds must be recalibrated for other platforms.

5.10 Summary of Findings

Four findings emerge from the evaluation.

First, speculative decoding is not automatically effective for offloaded MoE serving. Without caching and the corresponding execution-path optimizations, SD provides only 1.17× speedup over AR and the SD/AR ratio under UVA remains between 1.21–1.24× regardless of memory budget—turning SD from a powerful accelerator into a marginal improvement.

Second, the core role of expert caching is not merely improving hit rate, but *switching the effective bandwidth*: it converts expert access from PCIe-bound to HBM-bound, moving the system into a regime where SD can finally exploit its algorithmic advantage.

Third, caching alone is insufficient. Once the system enters the HBM-bound regime, scratchpad copy, CPU-GPU synchronization, residual misses, and kernel inefficiency emerge as the new critical-path bottlenecks. The ablation reveals that each Unblock optimization interacts with the acceptance rate in distinct ways: Pool-Direct slightly reduces acceptance (0.60→0.52) by changing the numerical path, but TASER recovers it and then some (0.52→0.67) through temporal routing stability. Cross-layer prefetch introduces mild cache pressure (0.67→0.61) but ensures coverage for cold-start scenarios. The combined Enable+Unblock stack achieves 4.80× over B1, with TASER improving acceptance above the uncached baseline. This demonstrates that the optimizations are complementary rather than competing.

Fourth, draft-guided preloading effectively maintains cache coverage under SD's expanded working set: at 24 GiB, the SD/AR ratio under BriskMoE reaches 1.76× despite the hit rate dropping from 80.4% to 64.8%, confirming that predictive prefetching compensates for the cache capacity shortfall and keeps SD effective across a broader range of GPU memory configurations.

These results explain why prior systems that address only one layer of the problem—caching without path optimization, or speculation without cache-awareness—are insufficient for latency-first single-request serving.

6 RELATED WORK

6.1 Speculative Decoding

Speculative decoding was introduced as a lossless acceleration mechanism for autoregressive language models by Leviathan et al. and Chen et al. [1, 12]. Subsequent work expanded the design space of draft models and verification strategies, including tree-based speculation, feature reuse, and dynamic depth control [13–16]. These works provide highly valuable techniques and mechanisms, but they largely assume that verification cost remains close to the cost of one target-model step—an assumption that is accurate for dense models but fails in our setting, because MoE verification expands the expert working set under CPU offloading.

6.2 MoE Inference and Offloading

Prior MoE systems have focused on distributed expert placement, multi-GPU execution, or offloading for autoregressive decoding [5, 7, 8, 18, 21]. These systems improved the practicality of large MoE models, but none of them identified the cache-speculation tension created by multi-token verification. In particular, MoE-Infinity and pre-gated MoE optimize single-token decode and therefore cannot explain why a cache that benefits AR may still harm SD.

6.3 MoE Plus Speculative Decoding

Recent work has started to combine MoE serving with speculative inference. Batch-oriented studies such as SpecMoE-Offload use Roofline-style reasoning and show that SD improves offloaded MoE throughput when the system loads a large expert set per batch [20]. Other approaches budget total expert activations, separate prefill and decode, or speculate on future expert activation [9, 17, 22]. However, these methods treat caching and speculation as largely orthogonal concerns. Unlike existing methods, our work actively studies their co-dependence: caching enables SD, while SD can also break caching. Moreover, BriskMoE addresses the operator-level cache path after the regime transition, not only the scheduling policy before it.

6.4 Operator-Level Inference Co-Design

Our work is also linked to the literature on operator-level inference co-design. Systems such as FlashAttention and PagedAttention improve model serving by redesigning memory movement rather than changing the model itself [2, 3, 10]. BriskMoE applies the same philosophy to offloaded MoE serving: once the model-level algorithm is fixed, most of the remaining wins come from controlling how expert data move through the memory hierarchy.

7 CONCLUSION

We have presented BriskMoE, a novel system that makes speculative decoding effective for single-request offloaded MoE inference. BriskMoE is developed with three novel designs: Enable switches expert access from PCIe to HBM via per-layer cache pools, Unblock removes the emerging cache-path bottlenecks through Pool-Direct, TASER, and Oracle Prefetch, and Protect maintains cache coverage under SD's expanded working set via draft-guided preloading. Our extensive experiments based on Qwen3-30B-A3B with EAGLE-3 confirm that BriskMoE achieves up to 4.80× speedup over

the offload baseline and restores SD to a 1.36× multiplier at 44 GiB (up to 2.29× at favorable memory budgets). As for future work, an immediate next step is to extend these techniques to multi-GPU distributed MoE serving and support quantized expert formats such as GPTQ and AWQ.

REFERENCES

[1] Charlie Chen, Sebastian Borgeaud, Geoffrey Irving, Jean-Baptiste Lespiau, Laurent Sifre, and John Jumper. 2023. Accelerating Large Language Model Decoding with Speculative Sampling. *arXiv preprint arXiv:2302.01318* (2023).

[2] Tri Dao. 2024. FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning. In *Proceedings of ICLR*.

[3] Tri Dao, Daniel Y. Fu, Stefano Ermon, Atri Rudra, and Christopher Re. 2022. FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness. In *Proceedings of NeurIPS*.

[4] Artyom Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238* (2023).

[5] Eldar Eliseev and Denis Mazur. 2023. Fast Inference of Mixture-of-Experts Language Models with Offloading. *arXiv preprint arXiv:2312.17238* (2023).

[6] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity. *Journal of Machine Learning Research* (2022).

[7] Changho Hwang, Wei Cui, Yifan Xiong, et al. 2023. Tutel: Adaptive Mixture-of-Experts at Scale. In *Proceedings of MLSys*.

[8] Sang-Yun Hwang, Kijun Yi, Giovanni Tarolli, and Nam Kim. 2023. Pre-Gated MoE: An Algorithm-System Co-Design for Fast and Scalable Mixture-of-Experts Inference. *arXiv preprint arXiv:2308.12066* (2023).

[9] Yifan Jin et al. 2024. SP-MoE: Efficient Serving of Mixture-of-Experts Models with Separated Prefill and Decode. *arXiv preprint* (2024).

[10] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, et al. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of SOSP*.

[11] Dmitry Lepikhin, HyoukJoong Lee, Yuanzhong Xu, et al. 2021. GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding. In *Proceedings of ICLR*.

[12] Yaniv Leviathan, Matan Kalman, and Yossi Matias. 2023. Fast Inference from Transformers via Speculative Decoding. In *Proceedings of the 40th International Conference on Machine Learning*.

[13] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE-2: Faster Inference of Language Models with Dynamic Draft Trees. In *Proceedings of EMNLP*.

[14] Yuhui Li, Fangyun Wei, Chao Zhang, and Hongyang Zhang. 2024. EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty. In *Proceedings of ICML*.

[15] Yuhui Li, Yuntian Zhong, et al. 2025. EAGLE-3: Scaling up Inference Acceleration of Large Language Models via Training-Free Speculative Sampling. *arXiv preprint* (2025).

[16] Xupeng Miao, Gabriele Oliaro, Zhihao Zhang, and Zhihao Jia. 2024. SpecInfer: Accelerating Generative LLM Serving with Tree-Based Speculative Inference and Verification. In *Proceedings of ASPLOS*.

[17] Kwangsoo Park et al. 2025. MoE-SpAc: Speculative Expert Activation for Efficient MoE Serving. *arXiv preprint* (2025).

[18] Samyam Rajbhandari, Conglong Li, Zhewei Yao, et al. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *Proceedings of ICML*.

[19] Stephen Roller, Sainbayar Sukhbaatar, Arthur Szlam, and Jason Weston. 2021. Hash Layers For Large Sparse Models. In *Advances in Neural Information Processing Systems*, Vol. 34. 17555–17566.

[20] SpecMoE-Offload. 2025. Speculative MoE Offloading. *arXiv preprint arXiv:2508.21706* (2025).

[21] Leyang Xue, Jiaao Zheng, Yao Li, et al. 2024. MoE-Infinity: Activation-Aware Expert Offloading for Efficient MoE Serving. *arXiv preprint arXiv:2401.14361* (2024).

[22] Seungwon Yi et al. 2025. MoE-Spec: MoE-Based Speculative Decoding with Expert-Aware Budgeting. *arXiv preprint* (2025).